Welcome to your next lesson: Checking the facts.

I'm getting your personalized lesson ready — using what you've learned, what's next, and the best way for you to practice.

Step by step, you're moving forward.

- Imagine you're printing a café receipt in Python ☕:

```python
total = 12.5
print("Your total is " + total)
```

Python will refuse this and throw a `TypeError`. The problem: `"Your total is "` is text, `12.5` is a number, and Python cares about these differences — they're called data types.

In your own words: does it make sense that Python needs to know what kind of thing a value is (text vs number), instead of treating everything the same?

Yes, that makes sense

Nice ?

If Python "cares" about what kind of value something is, we need a way to ask it, "Hey, what are you?". That's what `type()` is for.

Think of `type()` as a little label printer:

```python
price = 9.99
print(type(price))
```

This doesn't change `price` — it just shows Python's view of it (like "this is a number with decimals").

Before we try it in the editor:

If you see this in code:

```python
name = "Ron"
age = 42
```

which one do you expect to be a string (text): `name`, `age`, or both?

name

✅ Exactly — `name` is a string (text), and `age` is a number.

Now let's check that with `type()` like a detective ?♂

Here's the idea, step by step:

1. You create variables.
2. You ask Python what type they are using `type(...)`.
3. You print the answer so you can see it.

In the code editor, try this:

```python
```

```
name = "Ron"
age = 42

print(type(name))
print(type(age))
```

Run it first, make sure it works, then hit Submit code so I can see it.

```
Python
name = "Ron"
age = 42

print(type(name))
print(type(age))
```

Nice work, Ron ?

You just confirmed:

- `type(name)` → `<class 'str'>` → string (text)
- `type(age)` → `<class 'int'>` → int (whole number)

So `type()` tells us what Python thinks a value is.

Next tiny step:

If you have this:

```
Python
total = 12.5
```

what do you think `type(total)` will show: string, int, or float?

```
<class 'float'>
```

Exactly ✅

`12.5` is a float, so:

```
Python
total = 12.5
print(type(total))
```

will show:

```
Text
<class 'float'>
```

So far you've seen:

- `<class 'str'>` → text (string)
- `<class 'int'>` → whole number (integer)
- `<class 'float'>` → decimal number (floating-point)

Next step: we'll use `type()` to catch the error from this:

```
Python
total = 12.5
print("Your total is " + total)
```

Make sense so far about what `type()` does (just labels, doesn't change the value)?

Yes, it makes sense

Great ?

Let's use `type()` to see why that receipt line fails.

Step by step, in the code editor, write and run this:

```
Python
total = 12.5

print(type("Your total is "))
print(type(total))

print("Your total is " + total)
```

1. Hit Run (you should see the TypeError at the end).
2. Then hit Submit code so I can look at the output and walk through how to fix it.

```
Python
total = 12.5

print(type("Your total is "))
print(type(total))

print("Your total is " + total)
```

Perfect, Ron — this is exactly what I wanted to see ?

The output:

```
Text
<class 'str'>
<class 'float'>
TypeError: can only concatenate str (not "float") to str
```

So Python is basically saying:

- First line: `"Your total is "` → string
- Second line: `total` → float
- Error: "I can only glue string + string, not string + float."

To fix this, we need to convert the number `total` into text using `str()`.

Next step (one small change): in the editor, replace the last line with:

```
Python
print("Your total is " + str(total))
```

Run it, then hit Submit code so I can check it.

```
Python
total = 12.5

print(type("Your total is "))
```

```
print(type(total))

print("Your total is " + str(total))
```

Nice, Ron — that's the clean fix ✅

You just did two important things:

- Used `type()` to check what each value is.
- Used `str(total)` to convert a number (`float`) into text (`str`) so `"Your total is " + str(total)` works.

Quick picture in words:

- `total` is in the float box
- `str(...)` moves it into the string box so it can join other strings

Next tiny idea: `str()` turns things into text.

Now we'll look at the opposite: turning text that looks like a number into a real number with `int()` or `float()`.

Ready for that next step?

Yes, I'm ready

Cool ?

Let's flip the direction now.

Sometimes you get text that looks like a number, like:

```python
age_text = "42"
```

Right now, this is a string (text), not a real number. So:

```python
print(type(age_text))
```

will say `<class 'str'>`.

If you want to do math with it, you turn it into a real number with `int()`:

```python
age_text = "42"
age_number = int(age_text)

print(type(age_text))   # before
print(type(age_number)) # after
```

So `int("42")` converts `"42"` → `42` (string → int).

Does that basic idea make sense:

- `str(...)` → to text
- `int(...)` → to whole number?

Yes, that makes sense

Great ?

Let's make this concrete with a small price example.

Imagine you have a price stored as text:

```
price_text = "5.50"
```

Step by step, here's what we can do:

1. Check its type → `type(price_text)`
2. Convert it to a decimal number with `float(price_text)`
3. Add 2 to it, like tax or a tip

So in the editor, write and run this:

```
price_text = "5.50"

print(type(price_text))

price_number = float(price_text)

print(type(price_number))
print(price_number + 2)
```

Then hit Submit code so I can see the output.

```
price_text = "5.50"

print(type(price_text))

price_number = float(price_text)

print(type(price_number))
print(price_number + 2)
```

Nice, Ron — this is spot on ?

From your output:

- `type(price_text)` → `<class 'str'>` → it started as text
- `type(price_number)` → `<class 'float'>` → after `float(...)` it's a decimal number
- `price_number + 2` → `7.5` → you successfully did math with it

So now you've used all three:

- `str(x)` → to text
- `int(x)` → to whole number
- `float(x)` → to decimal number

Let's do one quick "string + number" fix to lock this in.

Imagine this line:

```
```

```
age = 7
print("Age: " + age)
```

Python will complain again (string + int).

To fix it, which conversion should you use on `age`?

I'll give you options next, and you can pick.

```
print("Age: " + int(age))
```

Nice try, Ron — you're thinking in the right direction ?

But remember what the error was about:

- Python doesn't like: string + int
- It does like: string + string

So in:

Python

```
age = 7
print("Age: " + age)
```

we don't want to turn `"Age: "` into a number — we want to turn `age` into text.

The fix should be:

Python

```
print("Age: " + str(age))
```

`str(age)` makes `age` a string, so we have: `"Age: " + "7"`.

Want to try this in the editor?

In the code editor, write and run:

Python

```
age = 7


print(type(age))
print("Age: " + str(age))
```

Then hit Submit code so I can confirm everything looks good.

```
oh right str(age) print("Age: " + str(age))
```

Exactly ?

That's the right fix:

Python

```
age = 7
print("Age: " + str(age))
```

Here's the pattern you now know:

- If you're joining with text → use `str(...)`
- If you need to do math with something that looks like a number in quotes → use `int(...)` or `float(...)`

Quick check-in, Ron:

Do you feel okay telling, in words, when you'd use each of these: `str()`, `int()`, `float()`, and `type()`?

If yes, I'll give you one small "receipt" challenge to finish this lesson strong.

Yes, I think so

Nice ? let's do that little "receipt" challenge and keep it simple.

Here's the goal:

Print this exact line using variables:

```
Total: 15.75
```

You must start with:

- a float for the number
- and then print it together with the word `"Total: "` without any `TypeError`.

So in the code editor, do this step by step:

1. Create the variable:
   `total = 15.75`
2. Use `type(total)` once to check what it is.
3. Print `"Total: 15.75"` using `total` and the right conversion so it doesn't crash.

Then hit Submit code and I'll check how you solved it.

Python

```
total = 15.75
print(type(total))
print("Total: " + str(total))
```

What do you think is the best fix here? ?

You have this code:

```
age = 7
print("Age: " + age)
```

It gives a `TypeError` because you can't add a string and an int.

Which fix is correct and simplest in this situation?

A.

```
print("Age: " + str(age))
```

B.

```
print("Age: " + int(age))
```

C.

```
print("Age: " + float(age))
```

D.

```
print(str("Age: " + age))
```

Nice, Ron — you nailed it ?

The best fix is:

Python

```
print("Age: " + str(age))
```

because:

- "Age: " is a string
- age is an int
- **str**(age) turns the number into a string, so Python is doing: string + string ✅

The others either still mix types or convert the wrong thing.

You've shown you can:

- Use **type**() to check what a value is
- Use **str**(), **int**(), and **float**() to convert between types and fix errors

You're in great shape on this lesson 😊

Want to move on to the next lesson, or do you want one more quick practice with types?

I'm ready to move on

Awesome work today, Ron 😊

You:

- Used **type**() to inspect what Python thinks a value is (str, int, float)
- Fixed the classic "text" + number TypeError by converting numbers to text with **str**(...)
- Turned text like "5.50" into real numbers with **int**() / **float**() so you could do math
- Printed a clean receipt line: Total: 15.75 without errors ✅

These are core skills you'll use all the time when working with real data (prices, ages, scores, etc.).

You're set to move on — go ahead and hit Next Lesson when you're ready, and I'll be right here for the next step 😊